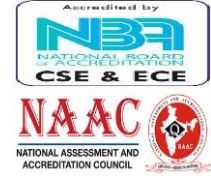




KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY
(UGC-Autonomous Institution)
Ghanpur(V), Ghatkesar(M), Medchal(D), Hyderabad-500088



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI&ML)

IOMP REVIEW – 02

MULTI-AGENT CODE REVIEW AND REFACTORING ASSISTANT

UNDER THE GUIDANCE OF :

G. DIVYA
ASSISTANT PROFESSOR
DEPARTMENT OF CSE (AI & ML)

PRESENTED BY :

K SRAVYA (23RA1A66B4)
N. PUJITHA (23RA1A6676)
K. YESHWANTH (23RA1A66D9)
P. VINITH (23RA1A66D2)

PROBLEM STATEMENT

- Continuous Code Review Required
- Code Quality Issues Increase
- Security Vulnerabilities Need Detection
- Complexity Affects Code Maintainability
- Manual Analysis Is Time-Consuming
- Refactoring Requires Developer Effort
- Testing Remains a Separate Task
- Need Integrated Multi-Agent Workflow

ABSTRACT

This project proposes a Multi-Agent Code Review and Refactoring Assistant, an AI-powered system that automates code review, refactoring, and test generation using specialized intelligent agents. A Code Reviewer Agent detects code quality issues, security vulnerabilities, and complexity problems using LLMs and tools such as Pylint and Bandit. A Refactoring Agent suggests optimized improvements, while a Test Generation Agent creates unit tests to validate the changes. Using LangGraph or CrewAI for agent coordination and human approval for modifications, the system aims to improve code quality, security, reliability, and developer productivity while supporting integration with modern CI/CD workflows.

LITERATURE SURVEY

1. CodeReviewer — Li et al. (2022)

- Pretrained model for automating code-review activities.
- Performs code-quality estimation, review-comment generation, and code refinement.
- Supports the **Code Reviewer Agent** of the proposed system.

2. ChatDev — Qian et al. (2024)

- Introduced a multi-agent framework for software development.
- Specialized LLM agents collaborate on tasks such as design, coding, and testing.
- Provides the **conceptual foundation for the multi-agent architecture**.

3. AutoCodeRover — Zhang et al. (2024)

- Uses LLMs, code search, and program structure for autonomous program improvement.
- Generates modifications to improve existing source code.
- Supports the **Refactoring Agent** in the proposed system.

4. CodeAgent — Tang et al. (2024)

- Multi-agent LLM framework for automated code review.
- Identifies vulnerabilities, inconsistencies, and coding-style violations.
- Supports automated **quality and security analysis** through specialized agents.

5. ChatUniTest — Chen et al. (2024)

- LLM-based framework for automated unit-test generation.
- Uses code context with generation, validation, and repair mechanisms.
- Supports the **Test Generation Agent** of the proposed system.

RESEARCH GAP :

- Existing studies mainly focus on **individual code-review, refactoring, multi-agent, or test-generation tasks**.
- The proposed system integrates these capabilities into a **single coordinated workflow** :
Code Review → Refactoring → Test Generation → Human Approval

EXISTING SYSTEM

- Static Code Analysis Tools
- Rule-Based Code Review
- LLM-Based Code Suggestions
- Automated Code Refactoring
- Automated Unit-Test Generation
- Specialized Review Agents
- Separate Maintenance Tools
- Limited Workflow Integration

DISADVANTAGES OF EXISTING SYSTEM

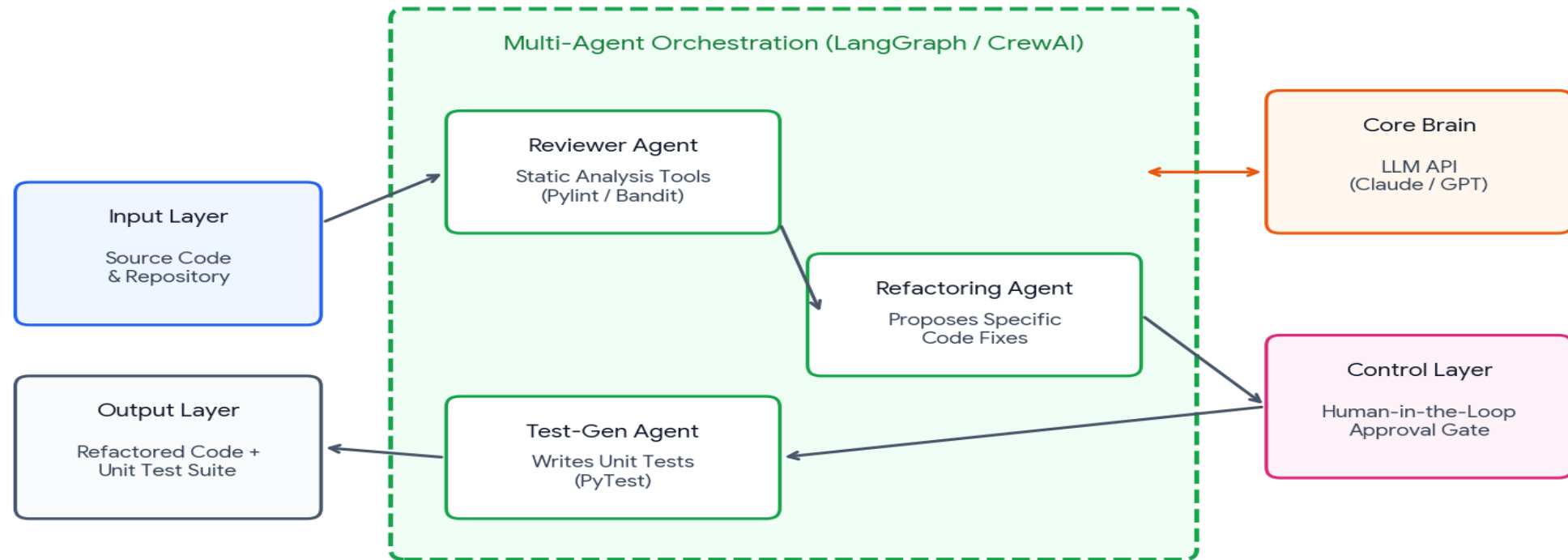
- Task-Specific Software Solutions
- Separate Review And Testing
- Limited Code Context Understanding
- AI Suggestions Need Verification
- Generated Tests Need Validation
- Increased Developer Intervention

PROPOSED SYSTEM

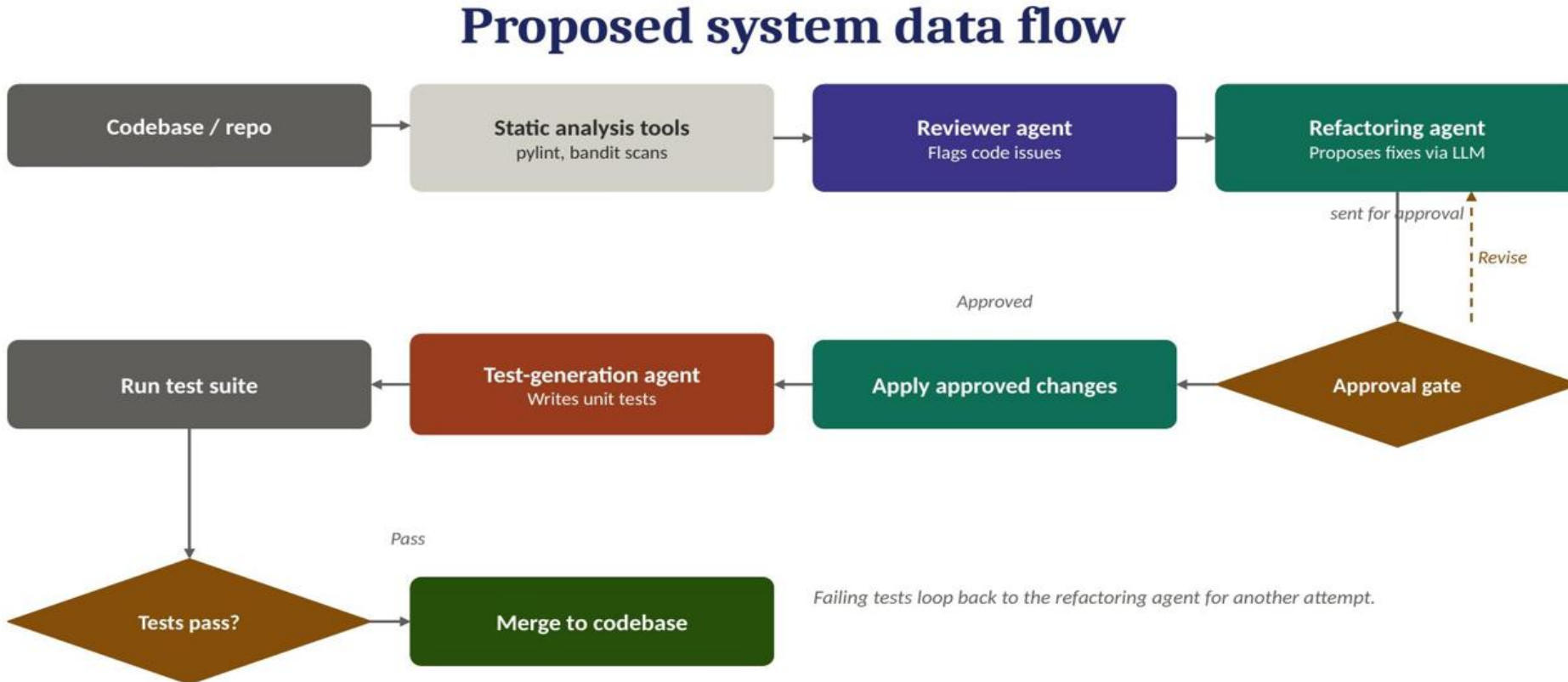
- Integrated Code Improvement Workflow
- Specialized AI Agents
- Automated Code Issue Detection
- AI-Based Refactoring Suggestions
- Human Approval Before Changes
- Automated Unit-Test Generation
- AI With Static Analysis
- Coordinated Agent Orchestration

PROPOSED SYSTEM ARCHITECTURE DIAGRAM

Proposed System Architecture



PROPOSED SYSTEM DATA FLOW DIAGRAM



METHODOLOGY

1. Code Input & Analysis Module

- Accepts source code from the selected repository.
- Parses and analyzes the code for structure and dependencies.
- Performs initial static analysis.

2. Code Review Module

- Identifies bugs, code smells, quality issues, and security vulnerabilities.
- Generates detailed review findings and improvement suggestions.

3. Refactoring Module

- Uses review findings to identify areas requiring improvement.
- Generates suitable refactoring suggestions for the existing code.

4. Test Generation Module

- Generates unit test cases for the reviewed/refactored code.
- Validates the generated tests and identifies potential issues.

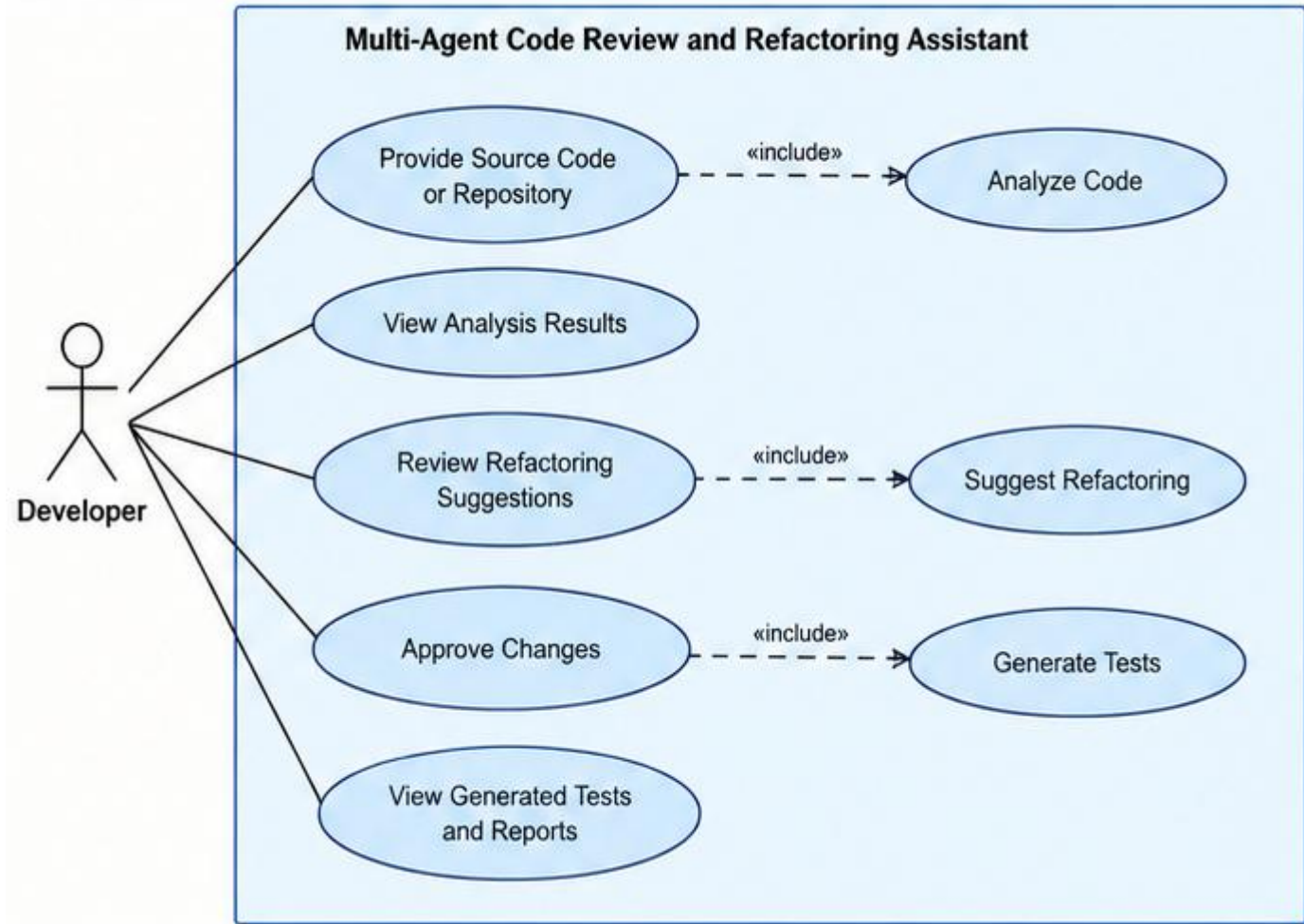
5. Validation & Human Approval Module

- Presents the proposed changes and generated tests for human verification.
- Applies approved changes and sends rejected changes back for revision.
- Produces the final improved code and unit-test suite.

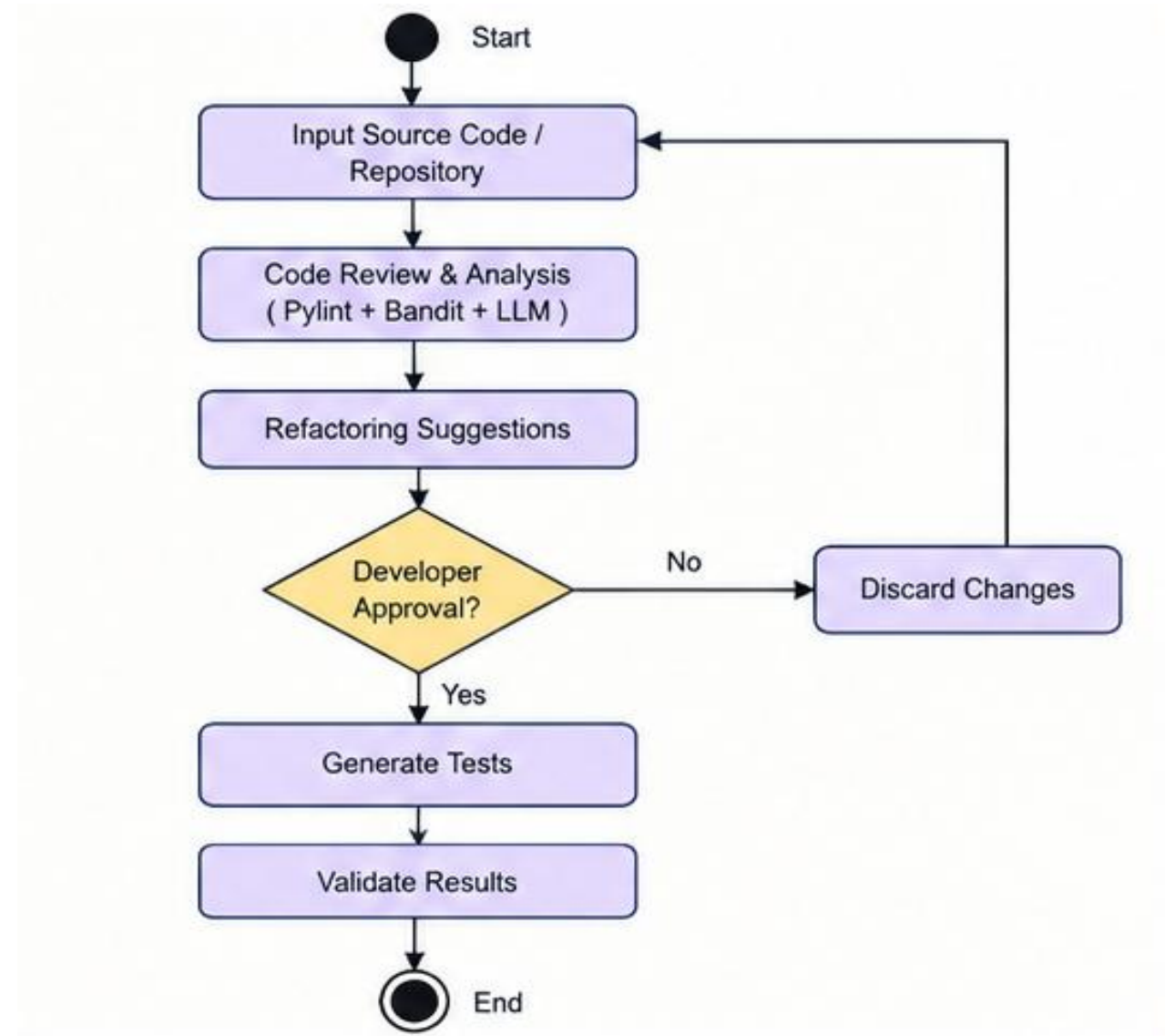
Input & Analysis → Code Reviewer Agent → Refactoring Agent → Test Generation Agent → Human Approval & Final Output

UML DIAGRAMS

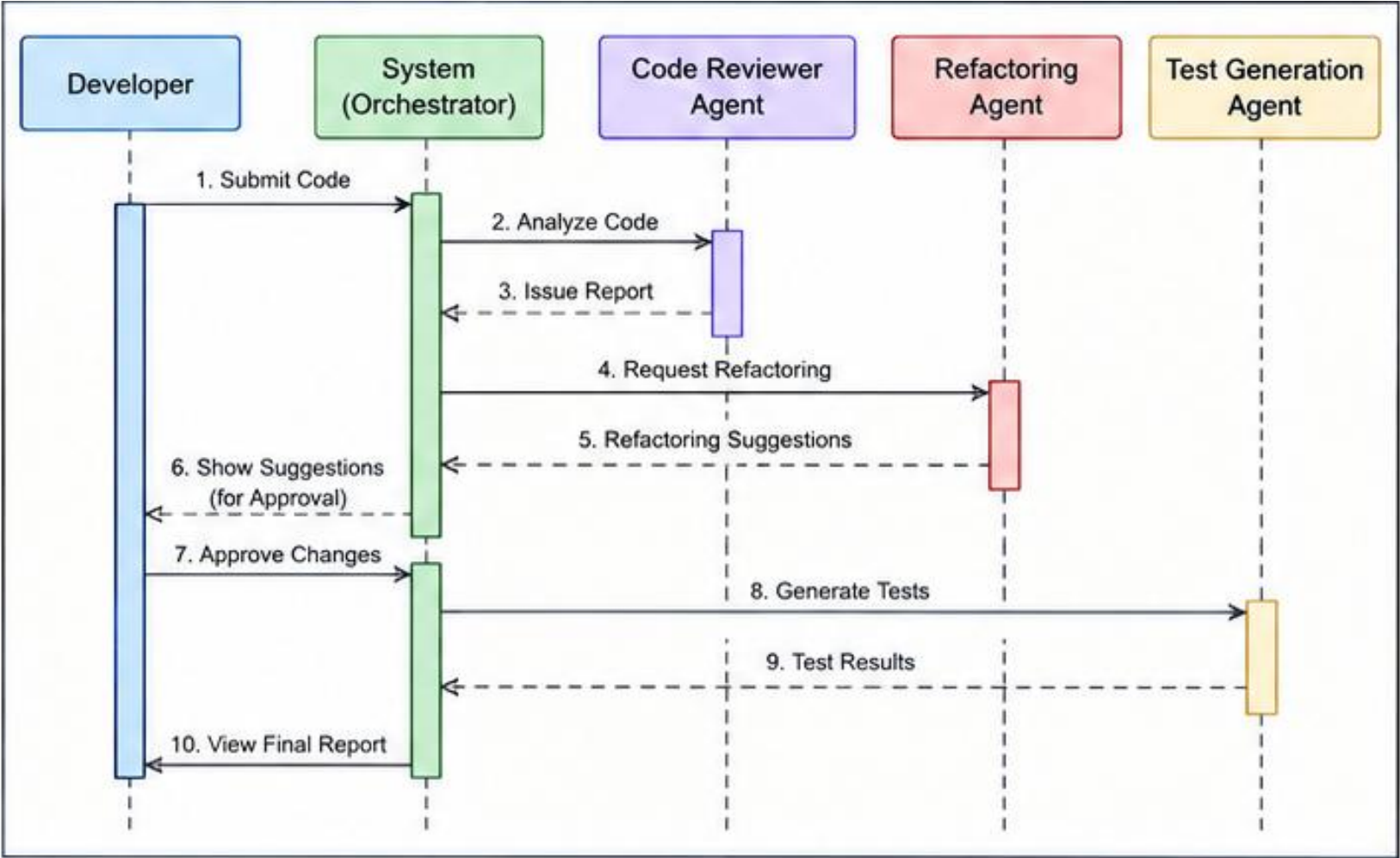
1. Use Case Diagram



2. Activity Diagram



3. Sequence Diagram



4. Component Diagram

